

Transformers and Dyck Languages: Error Detection, Correction, and Interpretability

Guillaume Wisniewski

guillaume.wisniewski@u-paris.fr

April 2026

This assignment is, in part, a direct response to a question raised by one of your distinguished predecessors, who enquired (with the measured scepticism that has come to define a certain strand of the student body) whether formal grammars retain any relevance in the age of LLMs, or whether they constitute, and I quote the wisest man I have ever heard, “a subject that died out roughly as fast as the dinosaurs, and at approximately the same time.”

The question is not entirely without merit. In the interest of intellectual honesty, I should admit that assembling a coherent answer required rather more effort than I had anticipated. So much effort, in fact, that by the time the material was in a fit state to be read, I found I had neither the energy nor the presence of mind to make any of it amusing. You will therefore find this document conspicuously short on wit. I can only apologise.

What I hope you will find, however, is that the underlying questions are genuinely interesting and that the practical component, which asks you to train and “interrogate” Transformer models in ways that are, if not entirely unheard-of, at least somewhat out of the ordinary, may go some way towards making the formal-language-theoretic scaffolding feel less like an archaeological curiosity and more like a living set of ideas. At the very least, it should be more diverting than a third week of parsing algorithms or a second week of computational semantics (this is, of course, humour).

Whether that is sufficient consolation is, I accept, a matter of taste.

You will also find four questions designated as bonus. I will be transparent: they exist primarily because the total number of questions I had drafted fell short of the figure I had set myself, and rounding up seemed the path of least resistance. I have no doubt whatsoever, however, that each and every one of you, possessed as you are of quite exceptional motivation, brilliance, and intellectual enthusiasm, will approach them with the rigour and creativity they deserve.

1 Background and Motivation

Dyck languages are the prototypical context-free languages. For $k \geq 1$, the Dyck language $D(k)$ is the set of well-balanced strings over k types of bracket pairs. Formally, let $\Sigma_k = \{a_1, \bar{a}_1, \dots, a_k, \bar{a}_k\}$,

where a_i denotes an opening bracket and $\bar{a}_i$ its matching closing bracket. Then $D(k)$ is defined by the grammar:

$$S \rightarrow \varepsilon \mid SS \mid a_i S \bar{a}_i \quad (1 \leq i \leq k).$$

A key theoretical result, you of course all known, is that every context-free language is a homomorphic image of $D(2) \cap R$ for some regular language R , making $D(2)$ *universal* for CFLs under homomorphism.

This assignment asks you to train a Transformer encoder to detect and correct errors in Dyck strings, and then to analyse what the model actually learns through attention visualisation and probing classifiers.

2 Data

2.1 Corpus generation

All data considered in this lab is generated synthetically. We write $D(k, n)$ for the subset of $D(k)$ whose strings have maximum nesting depth exactly n . Use $k = 2$ throughout (bracket pairs $()$, $[]$) unless stated otherwise. The table below specifies the recommended corpus sizes.

Split	Distribution	Lengths	Size
Train	$D(2, n \leq 4)$, balanced classes	4–40 tokens	50,000
Development	$D(2, n \leq 4)$, balanced classes	4–40 tokens	5,000
In-dist. test	$D(2, n \leq 4)$, balanced classes	4–40 tokens	5,000
OOD test	$D(2, n \in \{5, 6, 7\})$	40–80 tokens	5,000

Each example in the detection and correction splits is either a correct Dyck string or a corrupted version produced by applying *exactly one* of the following error operations at a uniformly random position:

- E1. Missing closer.** Delete a closing bracket.
- E2. Spurious opener.** Insert an extra opening bracket.
- E3. Type mismatch.** Replace a closing bracket with the closer of a different type (e.g. $(\dots]$ instead of $(\dots)$).
- E4. Premature close.** Insert a closing bracket before any matching opener exists on the stack.

2.2 Tokenisation

Use the following vocabulary:

$$\mathcal{V} = \{ (,), [,], [\text{PAD}], [\text{CLS}], [\text{SEP}] \}.$$

Prepend $[\text{CLS}]$ to every sequence and pad to a maximum length of 80 tokens.

3 Model

Use a **small Transformer encoder** (BERT-style). The intentionally modest size keeps attention matrices readable and training fast (< 10 minutes per run on a single GPU or a modern CPU, according to my meticulous calculations).

Hyperparameter	Recommended value
Number of layers	2–4
Number of attention heads	4
Hidden dimension d	64–128
Feed-forward dimension	$4d$
Dropout	0.1
Positional encoding	Sinusoidal (learned as bonus)
Maximum sequence length	80

4 Theoretical Aspects

1. Recall the Chomsky hierarchy. Where do Dyck languages sit within it? Sketch the argument showing that $D(2)$ is universal for context-free languages under homomorphism. Why does this make $D(k)$ an appropriate benchmark for testing structural generalisation?
2. Give two concrete examples from software engineering or computational linguistics where the relevant syntactic structure reduces to $D(k)$ for some $k \geq 1$. For each example, identify which of the four error types (E1–E4) corresponds to a common real-world mistake.
3. (bonus) An LSTM with hidden dimension h can recognise $D(1)$ in bounded memory, but requires memory growing with n to recognise $D(k)$ for $k \geq 2$ (where n is the nesting depth). Formulate the argument formally. Now consider a Transformer with fixed-width attention: what does this imply, in principle, about OOD generalisation to deeper nesting?

5 Training and in-distribution evaluation

4. Train a binary classifier using the [CLS] representation. Report accuracy, macro-F1, and a confusion matrix broken down by error type (E1–E4 vs. no error). Is any error type systematically harder to detect? Propose a linguistic or structural explanation.
5. Formulate correction as a sequence-labelling problem: for each token position t , predict a label in $\{\text{OK}, \text{DELETE}, \text{INSERT}(x), \text{REPLACE}(x)\}$ for $x \in \mathcal{V}$. Report token-level accuracy and exact-match accuracy (i.e. the fraction of strings for which the predicted correction fully restores the well-formed Dyck string). Discuss (at least two) limitations of this local formulation.

6. Plot training and development loss and accuracy against the number of epochs. Does the model converge cleanly? Identify any over-fitting and describe the regularisation strategies you applied.

6 Out-of-distribution generalisation

7. Evaluate the model trained on $D(2, n \leq 4)$ directly on the OOD test split ($n \in \{5, 6, 7\}$). Plot accuracy as a function of n and as a function of sequence length. Does performance degrade linearly, abruptly, or not at all?
8. Compare your model against a deterministic baseline: a pushdown automaton that implements the Dyck membership test exactly. This baseline is OOD-robust by construction. What does the comparison reveal about the nature of the representations learned by the Transformer?
9. Fine-tune the model on a small set of 500 examples with $n = 5$ only. Does accuracy on $n = 6$ and $n = 7$ improve? Does this suggest compositional generalisation or pattern memorisation? Justify your answer with quantitative evidence.

7 Attention analysis

10. For each attention head and each layer, visualise the average attention matrix over a set of 100 correct Dyck strings. Identify heads that appear to implement bracket matching (i.e. an opener at position i attends strongly to its paired closer at position j , and vice versa). Quantify this by computing, for each matched pair (i, j) :

$$\alpha_{i \rightarrow j} = A_{i,j} \quad \text{and} \quad \alpha_{j \rightarrow i} = A_{j,i},$$

where A is the attention matrix of the head under consideration, and reporting their mean and standard deviation over the test set.

11. Do the same matching heads remain active on erroneous strings? Describe qualitatively what happens to the attention pattern at and around the corrupted position. Formulate a hypothesis about the role of attention in localising the error.
12. (bonus) Compute *attention rollout* [1] to trace information flow from each input token to [CLS]. Do tokens involved in the error receive disproportionately large rollout weights?

8 Probing classifiers

13. **Global depth probe.** Train a linear regression from the final-layer [CLS] representation to the **maximum nesting depth** of the sequence (treated as a scalar). Report R^2 on the test

set. Then train a k -class linear classifier to predict depth as a discrete label ($n \in \{1, \dots, 7\}$) and report accuracy. Is nesting depth linearly encoded in [CLS]?

14. **Local depth probe.** For each token position t , the *current nesting depth* at position t is the number of unmatched openers in the prefix $s_1 \dots s_t$. Train a linear probe from the contextual representation of token t (at each layer) to predict this quantity. Plot probe R^2 as a function of layer depth. In which layer is local depth best encoded?
15. Compare probes trained on correct strings versus erroneous strings. Is depth correctly encoded up to the error position? What happens to the probe predictions after the corruption? Can you use probe predictions to localise the error without direct supervision?
16. (bonus) Train probes on intermediate layers and plot accuracy as a function of layer index for both the global and local depth tasks. Relate your observations to the notion of *progressive computation* in Transformer networks.

9 Open discussion

17. Based on all your experiments, do you conclude that the Transformer has *learned a rule* for Dyck membership, or that it has *memorised statistical patterns*? Construct your argument from the OOD results and the probing results.
18. Propose and implement *one* architectural or training modification aimed at improving OOD generalisation. Candidates include: relative positional encoding, curriculum learning (training on $n = 1$, then $n = 2$, ...), data augmentation by length, or a stack-augmented attention bias. Report the effect on the OOD test set.
19. To what extent are these results informative about the syntactic capabilities of large language models on natural language? Identify (at least two) important disanalogies between the Dyck task and natural language parsing that limit the scope of the conclusions.

10 Implementation Notes

I am about to provide you with the code for the function that generates strings according to a Dyck grammar and I do so purely for the pleasure of it, as it is a remarkably entertaining function to implement. I am, of course, perfectly aware that you are all sufficiently well-versed in algorithms and formal grammars to dash off your own implementation blindfolded. You are therefore more than welcome to stop reading at this point and submit your own solution, which will in all likelihood be considerably better than mine.

```
1 import random
```

```

2
3 PAIRS = [("(", ")"), ("[" , "]" )]
4
5 def generate_dyck(length: int, max_depth: int, k: int = 2) -> str | None:
6     """Return a random Dyck string of exactly `length` tokens (even),
7     with maximum nesting depth <= max_depth, or None on failure."""
8     assert length % 2 == 0
9     stack, result, depth = [], [], 0
10    remaining = length
11
12    for step in range(length):
13        must_close = (len(stack) == remaining)
14        can_open = (depth < max_depth) and (remaining > len(stack) + 1)
15
16        choices = []
17        if can_open and not must_close:
18            choices.append("open")
19        if stack:
20            choices.append("close")
21
22        if not choices:
23            return None
24
25        match random.choice(choices):
26            case "open":
27                pair = random.choice(PAIRS[:k])
28                stack.append(pair)
29                result.append(pair[0])
30                depth += 1
31            case "close":
32                pair = stack.pop()
33                result.append(pair[1])
34                depth = len(stack)
35
36    return "".join(result) if not stack else None

```

References

- [1] Abnar, S. and Zuidema, W. (2020). Quantifying attention flow in transformers. In *Proceedings of ACL 2020*, pages 4190–4197.
- [2] Bhattamishra, S., Ahuja, K., and Goyal, N. (2020). On the ability and limitations of transformers to recognise formal languages. In *Proceedings of EMNLP 2020*, pages 7096–7116.

- [3] Hewitt, J. and Manning, C. D. (2019). A structural probe for finding syntax in word representations. In *Proceedings of NAACL-HLT 2019*, pages 4129–4138.
- [4] Yao, S., Peng, B., Papadimitriou, C., and Narasimhan, K. (2021). Self-attention networks can process bounded hierarchical languages. In *Proceedings of ACL-IJCNLP 2021*, pages 3770–3785.